



Scraping Metrics in Prometheus: A Beginner's Guide

Introduction

What is Scraping in Prometheus?

Scraping is the process where **Prometheus Server** pulls metrics from configured targets. This allows Prometheus to **collect, store, and analyze time-series data**.

Why is Scraping Important?

- **Automates data collection** for monitoring applications and infrastructure.
 - **Ensures real-time insights** into system performance.
 - **Facilitates alerting and visualization** through PromQL and Grafana.
-

1. How Scraping Works in Prometheus

1.1 Scrape Targets

- Targets expose metrics on an **HTTP endpoint (/metrics)**.
- Prometheus **periodically queries these targets**.

Example of a metrics endpoint output:

```
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET"} 1024
http_requests_total{method="POST"} 256
```



1.2 Scrape Configurations

- Defined in the `prometheus.yml` configuration file.

Example scrape configuration:

```
scrape_configs:  
  - job_name: 'my_application'  
    scrape_interval: 15s  
    static_configs:  
      - targets: ['localhost:8080']
```

1.3 Scrape Interval and Timeout

- **scrape_interval**: Defines how frequently Prometheus scrapes metrics (default: 1m).
- **scrape_timeout**: Sets a timeout for the scrape request (default: 10s).

Example configuration:

```
scrape_configs:  
  - job_name: 'web_server'  
    scrape_interval: 30s  
    scrape_timeout: 5s  
    static_configs:  
      - targets: ['192.168.1.10:9100']
```

1.4 Service Discovery

- Prometheus can dynamically find targets in **Kubernetes, AWS, Docker Swarm, and Consul**.
- Avoids manually updating target lists.



Example Kubernetes service discovery:

```
scrape_configs:  
  - job_name: 'kubernetes'  
    kubernetes_sd_configs:  
      - role: endpoints
```

2. Configuring and Testing Scraping

Step 1: Install and Start Prometheus

```
wget  
https://github.com/prometheus/prometheus/releases/latest/download/promethe  
us-linux-amd64.tar.gz  
tar xvf prometheus-linux-amd64.tar.gz  
cd prometheus-linux-amd64  
./prometheus --config.file=prometheus.yml
```

Step 2: Check Scrape Targets

- Open **<http://localhost:9090/targets>** to verify active targets.

Step 3: Query Metrics with PromQL

- Open **<http://localhost:9090>**.
- Run a sample query:

```
http_requests_total
```



3. Know More: FAQs

1. How often does Prometheus scrape metrics?

- By default, every 1 minute (configurable via `scrape_interval`).

2. Can Prometheus scrape metrics from multiple services?

- Yes, multiple jobs can be defined in `prometheus.yml`.

3. What happens if a target is down?

- Prometheus marks it as **unreachable** but retains historical data.

4. Can Prometheus push metrics instead of pulling?

- No, but **Pushgateway** can be used for push-based metrics.

Conclusion

Scraping is **the backbone of Prometheus monitoring**. With proper configuration, Prometheus can **collect, process, and analyze real-time data** from various sources. By setting up **scrape intervals, service discovery, and querying with PromQL**, users can **gain deep insights into their system performance**.